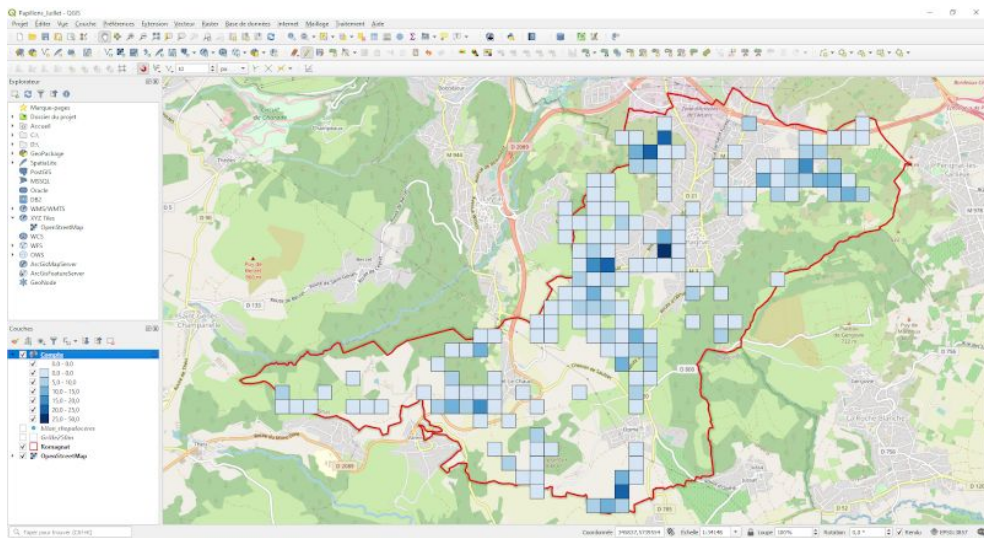
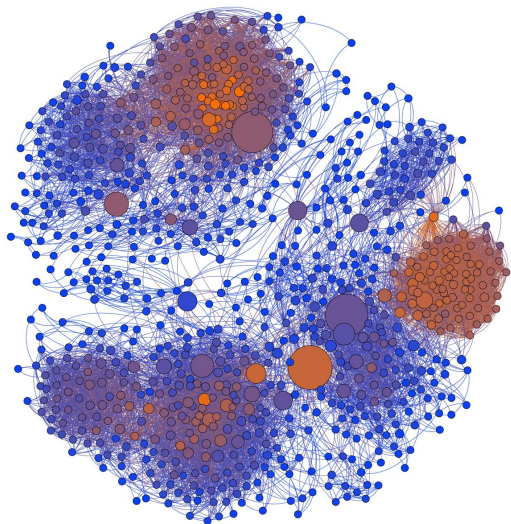


Initiation à Python

Calcul scientifique appliqué aux données archéologiques et géohistoriques
UMR 7041 - ArScAn







Automatisation de tâches répétitives
Inventaires
CSV / Excel
APIs / web scraping.

Analyse de données et statistiques :
traiter, manipuler et analyser
des jeux de données

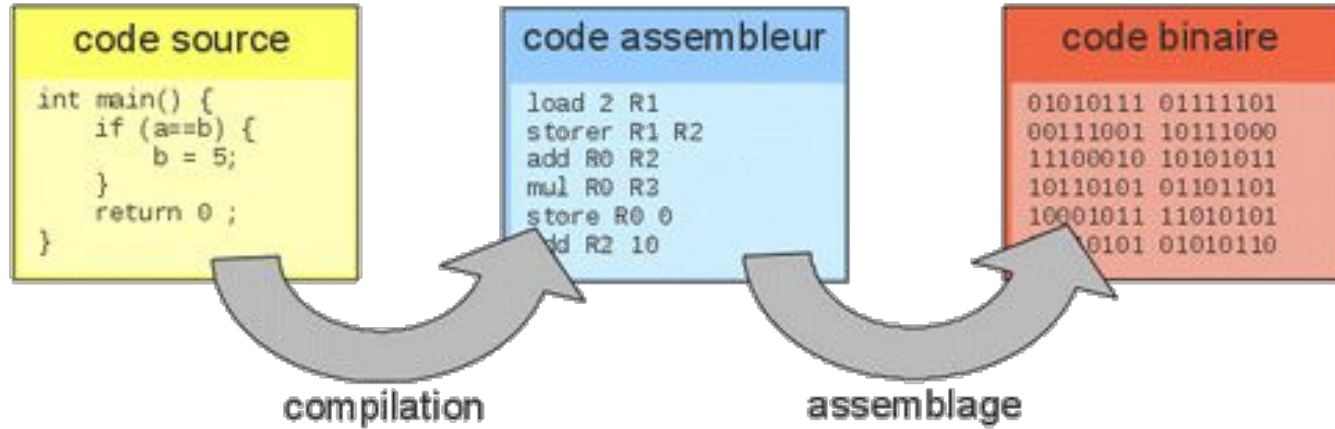


Programmation Web :
Développement de
sites web ou
d'applications web

Développement
d'outils pour la
gestion
documentaire

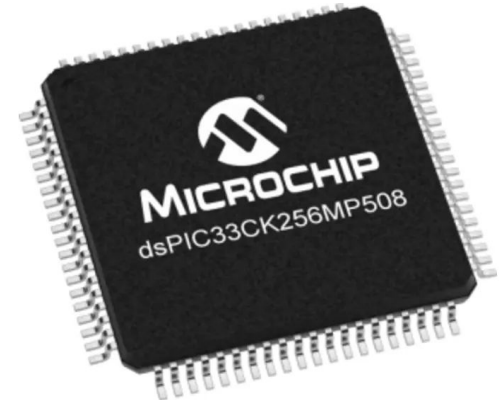
Manipulation et
analyse de fichiers
multimédias

Langages **compilés** / interprétés

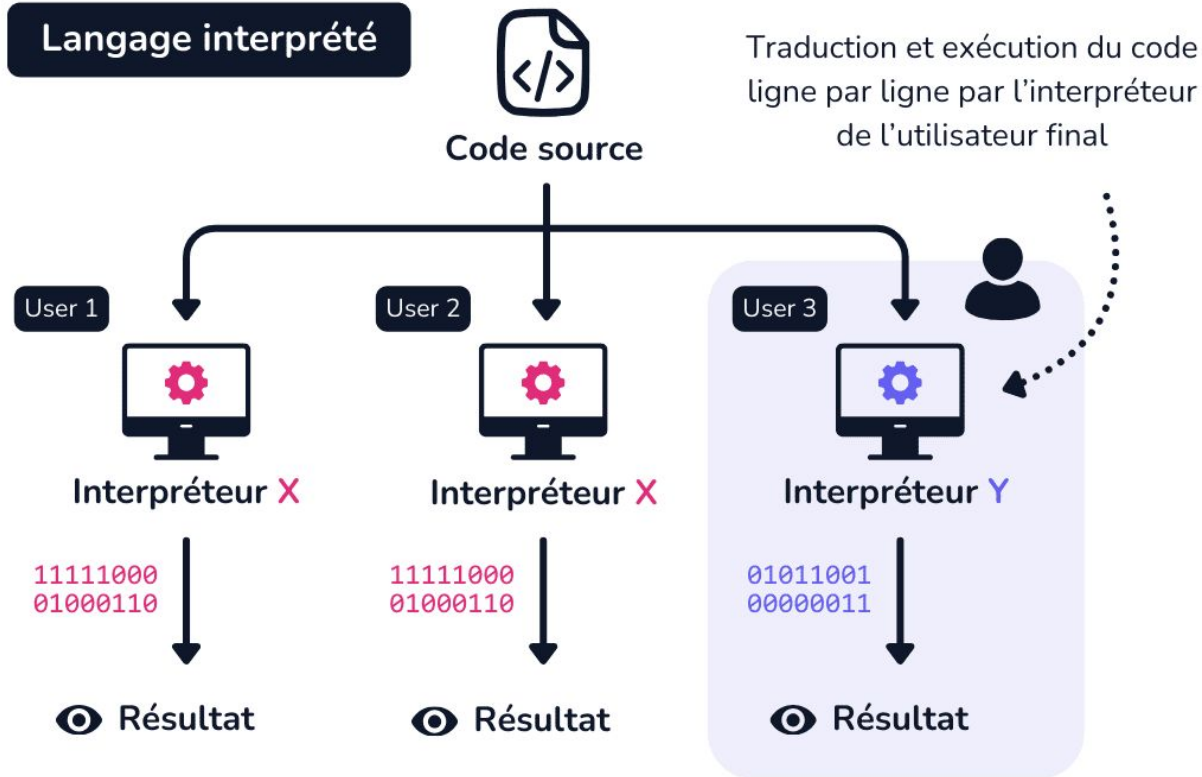


```
1 #include <stdio.h>  
2  
3 #define SUCCESS 0  
4  
5 int somme(int e1, int e2)  
6 {  
7     return (e1 + e2);  
8 }  
9  
10 int main(int argc, char const *argv[])  
11 {  
12     int a=4, b=5;  
13     printf("%d\n", somme(a, b));  
14     return SUCCESS;  
15 }
```

Avantages ?
Inconvénients ?



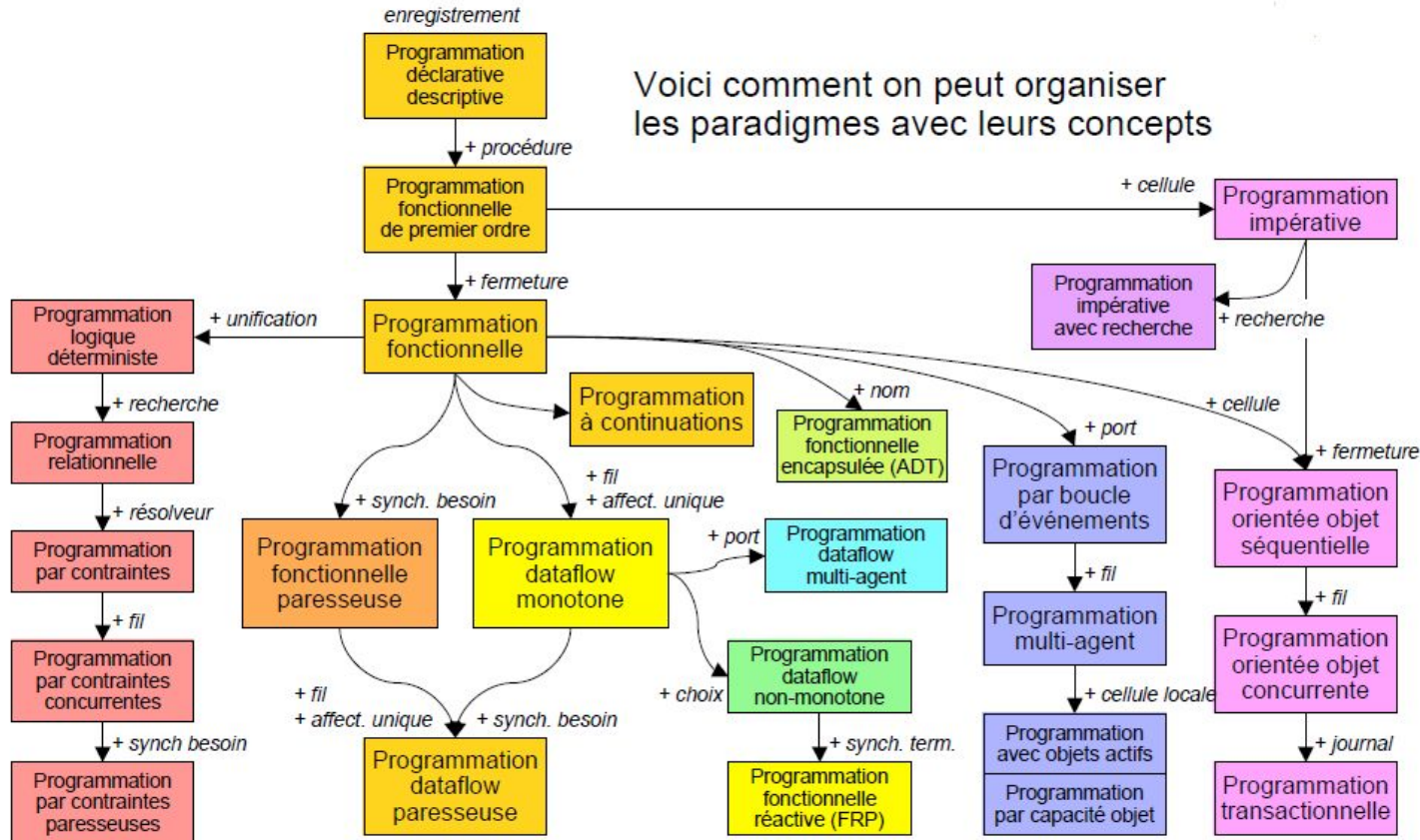
Langages compilés / interprétés



Avantages ?
Inconvénients ?

Paradigmes et langages de programmation

Voici comment on peut organiser les paradigmes avec leurs concepts



Paradigmes et langages de programmation

Paradigm	Languages
Functional	Erlang, F#, Haskell, Lisp, Ocaml, Perl, Racket, Ruby, Rust;
Imperative	Ada, C, C++, F#, Fortran, Go, Ocaml, Pascal, Rust;
Object-Oriented	Ada, C++, C#, Chapel, Dart , F#, Java, JavaScript, Ocaml, Perl, PHP, Python, Racket, Rust, Smalltalk, Swift, TypeScript;
Scripting	Dart, Hack, JavaScript, JRuby, Lua, Perl, PHP, Python, Ruby, TypeScript;

Un langage de script est un langage de programmation interprété (sans compilation explicite), conçu pour manipuler les fonctionnalités d'un système informatique. Sa syntaxe de haut niveau permet de s'affranchir des contraintes des commandes de bas niveau. Ses capacités dépendent de l'environnement et de l'interface de son interpréteur.

La **programmation fonctionnelle** est un paradigme de programmation de type déclaratif.

Elle conçoit le calcul comme l'évaluation de fonctions mathématiques.

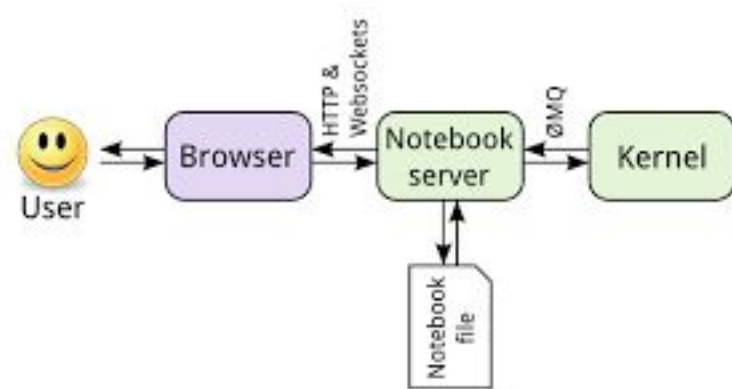
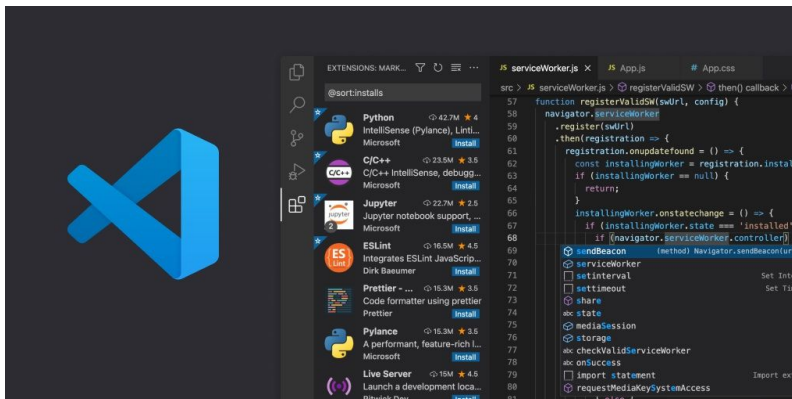
La **programmation impérative** est un paradigme informatique qui définit les opérations par des séquences d'instructions. Ces instructions sont exécutées par l'ordinateur afin de modifier l'état du programme.

C'est la forme de programmation la plus courante. Elle s'oppose à la programmation déclarative, qui inclut notamment la programmation logique et la programmation fonctionnelle.

Le paradigme de la **programmation orientée objet**, ou programmation par objet, est une approche fondamentale de la programmation informatique :

- Un objet modélise une entité du monde réel (ex : voiture, personne) ou conceptuelle.
- Représenter ces objets et leurs relations.
- Leur interaction permet de concevoir les fonctionnalités et d'optimiser la résolution des problèmes.

Exécution de programmes Python : local vs. distant



IDE : integrated development environment

VS code

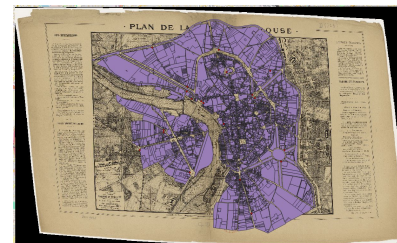
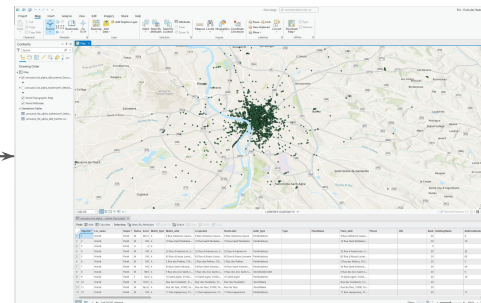
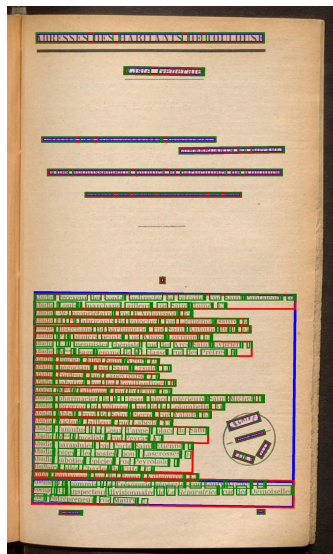
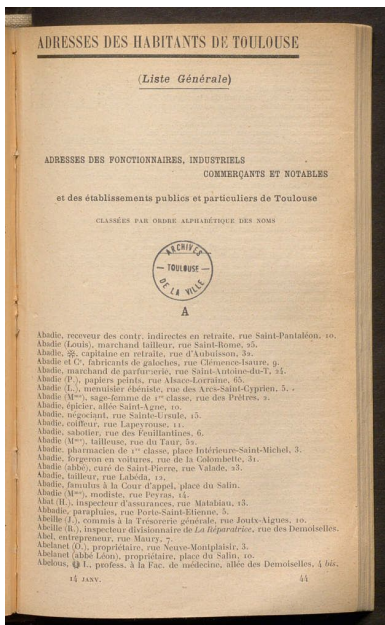
PyCharm

Eclipse

...

Dans un navigateur ?

Exemple 1 : Annuaire Toulouse 1900 - OCR, nettoyage, géocodage



Exemples 2 :
Applications web PTM



Interrogation d'une base de données avec des expressions régulières

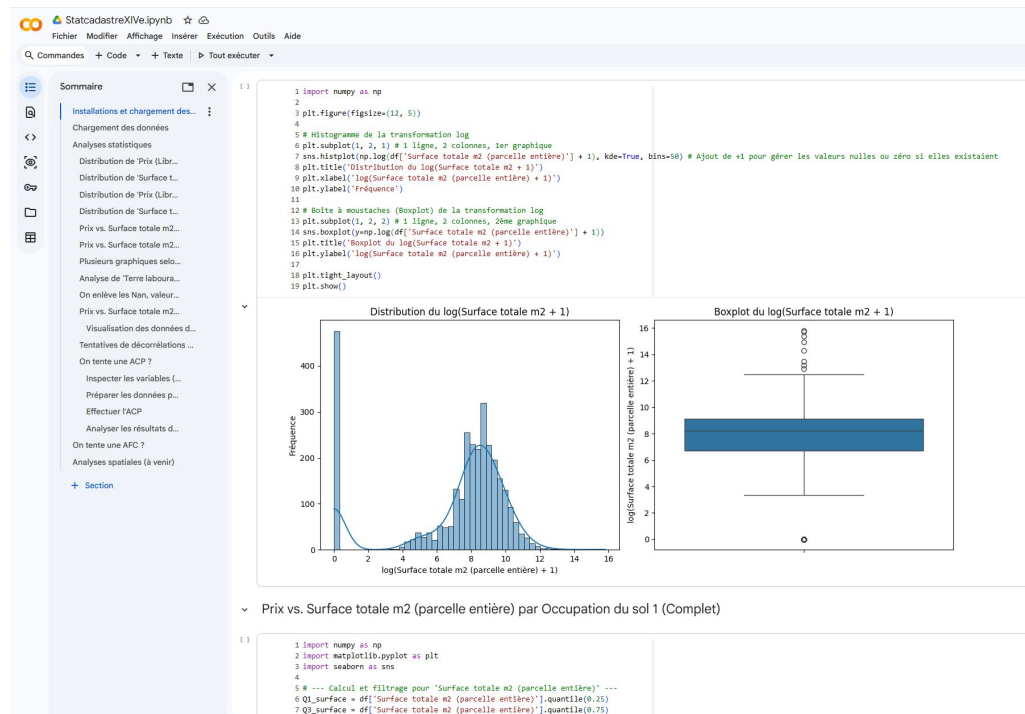


Géoréférencement de cartes anciennes et tuilage



Calcul de plus courts chemins

Exemples 3 : Carnet / Notebook Jupyter



Analyses statistiques & spatiales / collaboratives

https://colab.research.google.com/drive/1F3BmNGHl6nZj2D-7_aGPeOfEfdonmV

Publi : <https://jhnr.net/articles/10.25517/jhnr.v10i1.86>

Exemple 4

Cemantix Breaker



spaCy

Industrial-Strength
Natural Language
Processing

<https://spacy.io/>

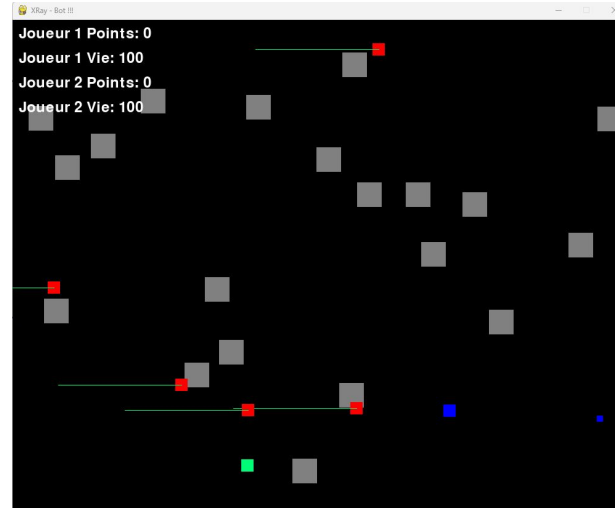
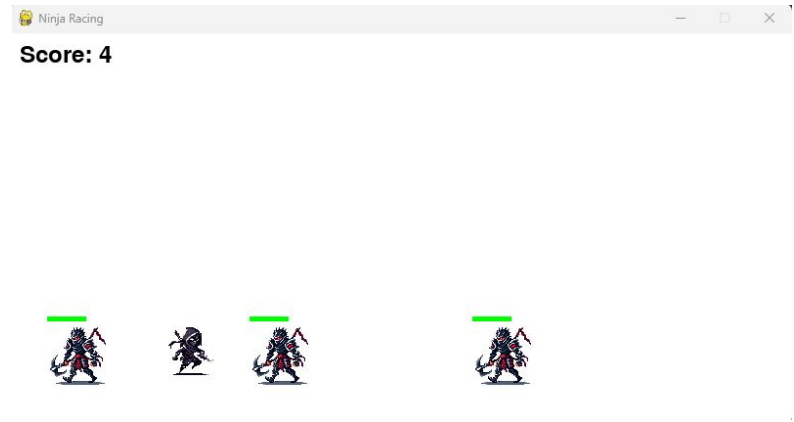
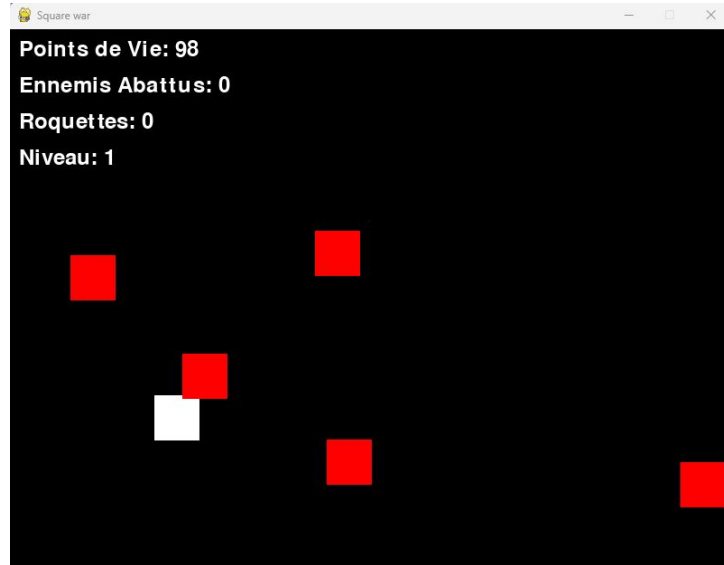
The screenshot shows the Cemantix Breaker game interface. At the top, there's a word cloud for 'cemantix' and a smaller one for 'pedantix'. Below the word cloud, it says 'Jour n°1038' and 'Trouvé par 11996 personnes'. A progress bar shows the percentage of words guessed, with a temperature gauge icon. The bar is currently at 100.00%. Below the bar, it says 'Le mot d'hier était os' and 'cemantix'. A green box contains the message 'Bravo ! Vous êtes 11996° à le trouver, en 47 coups. Résumé du jour :'. Below this, there's a 'Partagez et revenez jouer demain.' button and a 'Voir les mots les plus proches' button. At the bottom, there's a table of words to guess.

N°	Mot	°C	% Progression
47	indice	100,00	1000 BINGO !
20	valeur	38,20	989
43	cotation	35,52	976
42	estimation	33,25	948
46	chiffre	32,06	932
6	niveau	29,74	888
18	volume	27,37	827
37	signification	24,25	653

<https://cemantix.certitudes.org/>

Exemple 5

Jeux vidéo



Route 1 : installer et configurer VS Code

The screenshot displays the Visual Studio Code (VS Code) interface with a dark theme. The Explorer sidebar on the left shows a project structure with folders like 'PYTHON SIS' and '1_manipulations_basiques.py'. The main editor area is open to a file named '1_manipulations_basiques.py', which contains Python code for creating and writing to a CSV file. The code includes comments in French and uses the 'csv' module. The output console at the bottom shows the execution results, including the message 'Fichier CSV créé !'. On the right side, there is a 'CONVERSATION' panel with a 'RECENT SESSIONS' list and a 'Build with Agent' section. The 'Build with Agent' section includes a note about response accuracy and a link to 'Generate Agent Instructions'. At the bottom right, there is a 'Cellule 50 sur 65' indicator and a 'Continue (IN)' button.

```
import csv

# Ecrivez dans un fichier CSV
with open('csv/nouveau_fichier.csv', 'w', newline='') as fichier_csv:
    ecrivain_csv = csv.writer(fichier_csv)
    ecrivain_csv.writerow(['nom', 'age', 'ville'])
    ecrivain_csv.writerow(['Alice', 25, 'Paris'])
    ecrivain_csv.writerow(['Bob', 30, 'Lyon'])

print("Fichier CSV créé !")
```

Fichier CSV créé !

Exercices sur les fichiers

Exercice 1 : Boucle et fichier texte

Créez un programme qui lit le fichier texte `mails_exemple.txt` ligne par ligne et affiche chaque ligne à l'écran. Si le fichier contient le mot "important", affichez une alerte particulière.

Exercice 2 : Manipulation de CSV

Créez un programme qui lit un fichier CSV contenant une liste de produits (nom, prix, quantité) et qui calcule la valeur totale des stocks (prix * quantité).

```
# Exercice 1 : Lecture avec recherche de mot
```

```
# Exercice 2 : Calcul de valeur totale
```

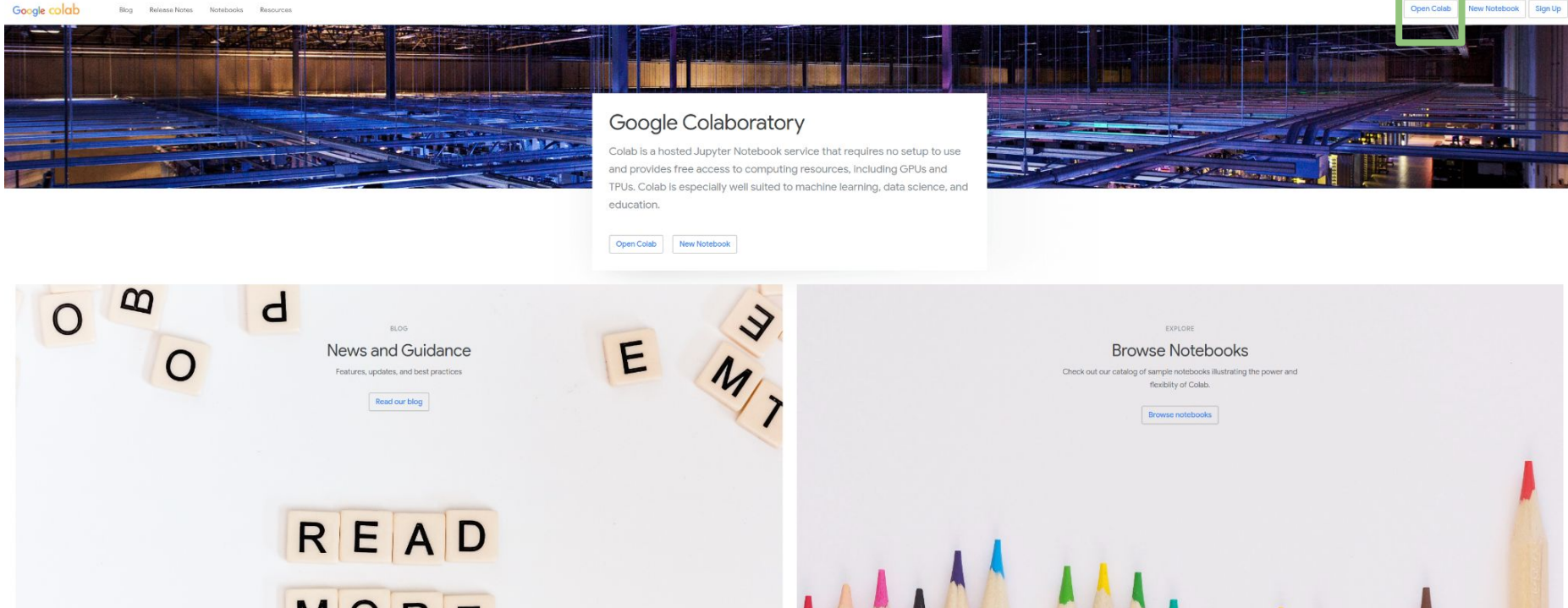
Résumé

Dans cette première séance, nous avons couvert :

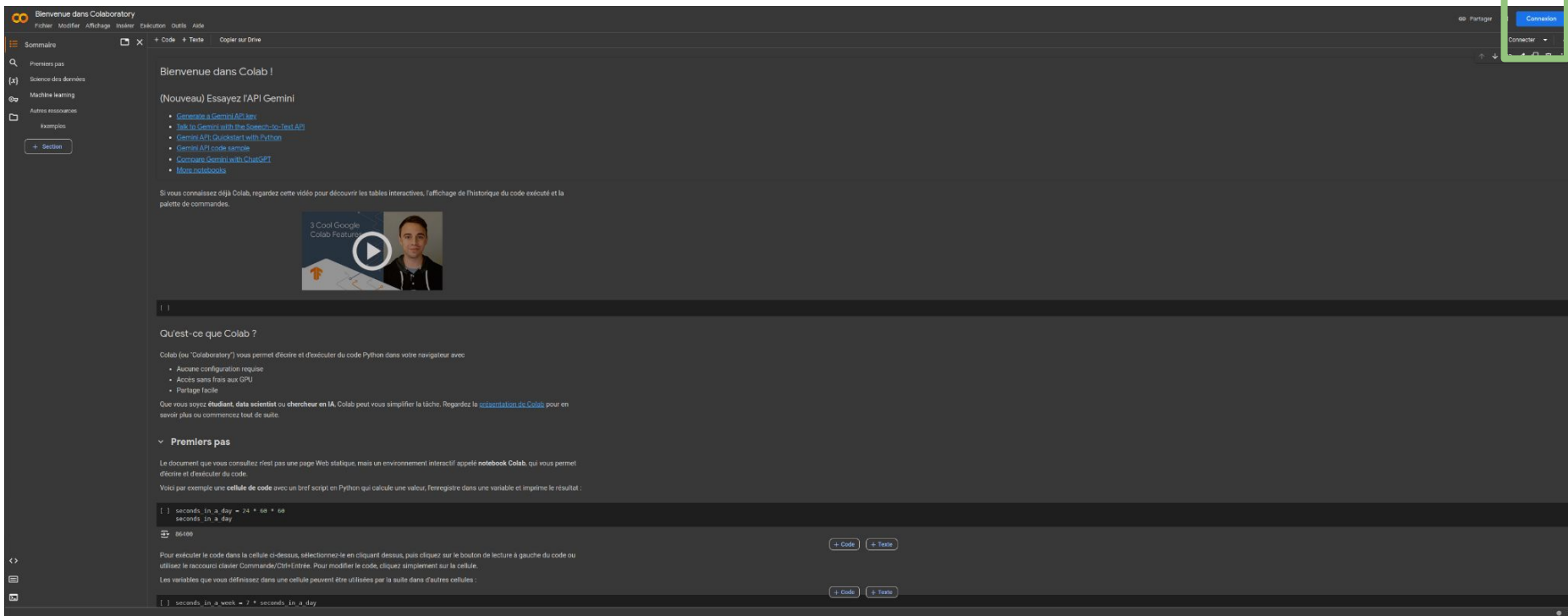
1. **Introduction à la programmation** : langages compilés vs interprétés
2. **Variables et types** : int, float, str, bool
3. **Opérations** : arithmétiques, booléennes, sur les chaînes
4. **Formatage** : f-strings et slicing
5. **Conditions** : if/elif/else
6. **Boucles** : for, while, enumerate, zip
7. **Fonctions** : définition, paramètres, retour
8. **Gestion d'erreurs** : try/except

Route 2 : Google Colab

Aller à l'adresse : <https://colab.google/>



Connexion avec un compte gmail



Bienvenue dans Colaboratory

Fichier Modifier Affichage Insérer Exécution Outils Aide

Sommaire

Prémiers pas

Science des données

Machine learning

Autres ressources

Exemples

+ Section

Bienvenue dans Colab !

(Nouveau) Essayez l'API Gemini

- [Introduction à Gemini API](#)
- [Hello to Gemini with the Jupyter-Notebook API](#)
- [Gemini API Quickstart with Python](#)
- [Gemini API code sample](#)
- [Compare Gemini with ChatGPT](#)
- [More notebooks](#)

Si vous connaissez déjà Colab, regardez cette vidéo pour découvrir les tables interactives, l'affichage de l'historique du code exécuté et la palette de commandes.

3 Cool Google Colab Features

[]

Qu'est-ce que Colab ?

Colab (ou "Colaboratory") vous permet d'écrire et d'exécuter du code Python dans votre navigateur avec :

- Aucune configuration requise
- Accès sans frais aux GPU
- Partage facile

Que vous soyez **étudiant**, **data scientist** ou **chercheur en IA**, Colab peut vous simplifier la tâche. Regardez la [représentation de Colab](#) pour en savoir plus ou commencez tout de suite.

▼ Premiers pas

Le document que vous consultez n'est pas une page Web statique, mais un environnement interactif appelé **notebook Colab**, qui vous permet d'écrire et d'exécuter du code.

Voici par exemple une **cellule de code** avec un bref script en Python qui calcule une valeur, l'enregistre dans une variable et imprime le résultat :

```
[ ] seconds, in a day = 24 * 60 * 60
seconds, in a day
```

86400

Pour exécuter le code dans la cellule ci-dessus, sélectionnez-le en cliquant dessus, puis cliquez sur le bouton de lecture à gauche du code ou utilisez le raccourci clavier Commande/Ctrl+Entrée. Pour modifier le code, cliquez simplement sur la cellule.

Les variables que vous définissez dans une cellule peuvent être utilisées par la suite dans d'autres cellules :

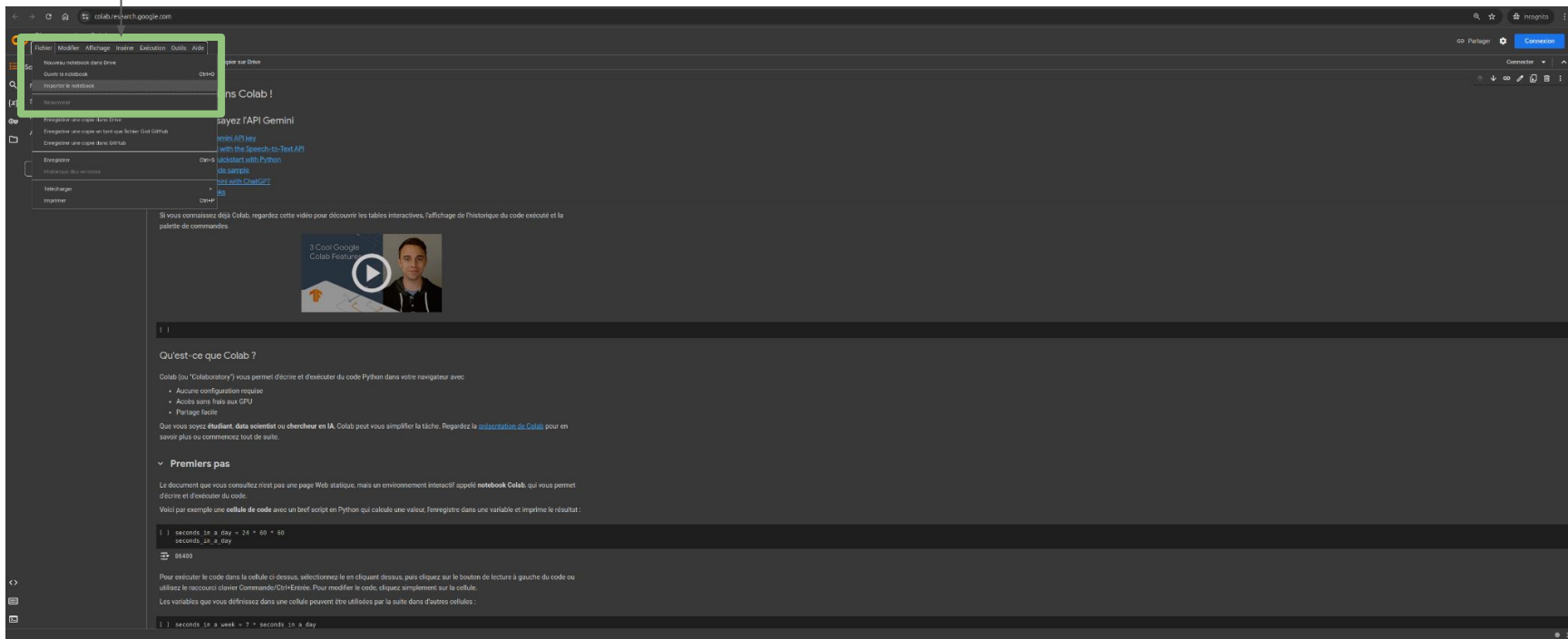
```
[ ] seconds, in a week = 7 * seconds, in a day
```

+ Code -+ Texte

+ Code -+ Texte

Connecter

Fichier / Importer le notebook



The screenshot shows the Google Colab web interface. The 'Fichier' (File) menu is open, and the 'Importer le notebook' (Import notebook) option is highlighted. The interface is in French. The main content area displays a welcome message and a video player. Below the video, there is a section titled 'Qu'est-ce que Colab ?' (What is Colab?) which explains that Colab is an interactive environment for running Python code. It lists features such as no configuration required, free access to GPUs, and easy sharing. A 'Premiers pas' (First steps) section follows, explaining that the document is an interactive notebook and providing instructions on how to execute code cells. The code cell shown contains a simple calculation: `seconds_in_a_day = 24 * 60 * 60` and `seconds_in_a_week = 7 * seconds_in_a_day`.

colab.research.google.com

Fichier Modifier Affichage Insertion Extension Outils Aide

Nouveaux notebooks dans Drive

Ouvrir le notebook

Importer le notebook

Rechercher

Envoyer une copie dans Drive

Envoyer une copie en tant que fichier Google Docs

Envoyer une copie dans VM Lab

Envoyer

Historique des versions

Télécharger

Imprimer

Connecter

Connection

ins Colab !

sayez l'API Gemini

with API key

with the Speed-to-Test API

kickstart with Python

de sample

not with ChatGPT

Si vous connaissez déjà Colab, regardez cette vidéo pour découvrir les tables interactives, l'affichage de l'historique du code exécuté et la palette de commandes.

3 Cool Google Colab Features

Qu'est-ce que Colab ?

Colab (ou "Colaboratory") vous permet d'écrire et d'exécuter du code Python dans votre navigateur avec :

- Aucune configuration requise
- Accès sans frais aux GPU
- Partage facile

Que vous soyez étudiant, data scientist ou chercheur en IA, Colab peut vous simplifier la tâche. Regardez le [tutorial de Colab](#) pour en savoir plus ou commencez tout de suite.

▼ Premiers pas

Le document que vous consultez n'est pas une page Web statique, mais un environnement interactif appelé **notebook Colab**, qui vous permet d'écrire et d'exécuter du code.

Voici par exemple une **cellule de code** avec un bref script en Python qui calcule une valeur, l'enregistre dans une variable et imprime le résultat :

```
1 | seconds_in_a_day = 24 * 60 * 60
   seconds_in_a_day
   86400
```

Pour exécuter le code dans la cellule ci-dessus, sélectionnez-le en cliquant dessus, puis cliquez sur le bouton de lecture à gauche du code ou utilisez le raccourci clavier Commande/Ctrl+Entrée. Pour modifier le code, cliquez simplement sur la cellule.

Les variables que vous définissez dans une cellule peuvent être utilisées par la suite dans d'autres cellules :

```
1 | seconds_in_a_week = 7 * seconds_in_a_day
```


Dans un autre onglet, aller à l'adresse :
<https://github.com/ericmermet/python-shs>

The screenshot shows the GitHub repository page for 'python-tut' by user 'ericmermet'. The repository is public and has 1 branch and 0 tags. The main branch is 'main'. The repository contains several files and folders, including 'csv', 'Python1_tut_INNO3.ipynb', 'maia_examples.txt', and 'nouveau_fichier.txt'. The 'README' section is currently empty, with a prompt to 'Add a README'. The right sidebar shows sections for 'About', 'Releases', 'Packages', and 'Languages'. The 'Languages' section shows 'Jupyter Notebook' at 100.0%.

ericmermet / python-tut

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

python-tut PUBLIC

main 1 Branch 0 Tags

Go to file Add file Code

ericmermet Delete correction_exercice_final.ipynb 614730 25 minutes ago 6 Commits

csv data structures / example files 7 hours ago

Python1_tut_INNO3.ipynb add final exercice 27 minutes ago

maia_examples.txt data structures / example files 7 hours ago

nouveau_fichier.txt data structures / example files 7 hours ago

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

About

No description, website, or topics provided.

Activity

0 stars

1 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

Jupyter Notebook 100.0%

© 2024 GitHub, Inc. Terms Privacy Security Status Docs Contact Manage cookies Do not share my personal information

Revenir sur le google collabs et entrer l'adresse du git dans la barre

The image shows a Python IDE interface with a document titled "Qu'est-ce que la programmation ?". The document content is as follows:

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Manipuler des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- **Langages de bas niveau** : proches du matériel, comme l'assembleur.
- **Langages de haut niveau** : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant de pouvoir être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

The modal window "Ouvrir le notebook" is open, showing a search bar with the URL "https://github.com/enicomet/python-ut" and a list of recent repositories, including "python-ut" and "python-ut-jupyter".

Revenir sur le google collabs et entrer l'adresse du git dans la barre

Python LUT - JupyterLab

Fichier Modifier Affichage Insérer Extension Outils Aide [Intégration d'extensions des modifications](#)

Fichiers

Code Texte Copier sur Drive

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Gérer des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- Langages de bas niveau : proches du matériel, comme l'assembleur.
- Langages de haut niveau : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant d'être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

Ouvrir le notebook

Exemples > Recherchez une URL, GitHub ou effectuez une recherche par organisation ou par <https://github.com/enicemmet/python-lut> ☐ Inclure les dépôts privés

Récents >

Dépot: ☐ Branche:

Google Drive >

GitHub >

Importer >

Python_LUT_RNCS.jar

0 s terminée à 17:55

Revenir sur le google collabs et entrer l'adresse du git dans la barre

Python LUT_INNO3.py

Fichier Modifier Affichage Index Exécution Outils Aide [Importation d'un nouveau module](#)

Fichiers

- sample_data
- main_examples.txt

Qu'est-ce que la programmation ?

La programmation consiste à écrire des instructions pour qu'un ordinateur effectue des tâches spécifiques. Ces instructions sont écrites dans des langages de programmation, qui sont des systèmes de communication entre l'humain et la machine.

Un programme est une suite d'instructions que l'ordinateur va exécuter dans un ordre logique pour résoudre un problème ou accomplir une tâche.

Les programmes informatiques permettent de :

- Automatiser des tâches répétitives.
- Traiter des données (comme des textes, des nombres, des images, etc.).
- Communiquer avec des services en ligne (API).
- Gérer des fichiers et des systèmes d'information.

Les langages de programmation permettent aux programmeurs de s'abstraire de la complexité du matériel informatique et d'écrire du code de manière plus lisible et productive. Parmi les langages les plus connus, on peut citer Python, C, Java, JavaScript, et bien d'autres.

Langages de programmation

Les langages de programmation peuvent se diviser en plusieurs catégories, dont :

- Langages de bas niveau : proches du matériel, comme l'assembleur.
- Langages de haut niveau : plus abstraits, faciles à lire et à comprendre pour les humains (comme Python, Java, ou C++).

Les langages de programmation ne diffèrent aussi selon la manière dont ils sont exécutés par la machine. Deux grandes catégories se distinguent : les langages **compilés** et les langages **interprétés**.

Langages compilés vs Langages interprétés

Lorsque nous écrivons un programme, l'ordinateur ne peut pas comprendre directement les instructions que nous avons écrites en tant que développeur. Il a besoin de les traduire en instructions que le processeur peut exécuter (du langage machine, composé de 0 et de 1).

Langages compilés

Les **langages compilés** nécessitent un processus de compilation avant d'être exécutés. Cela signifie que le programme est converti une seule fois en un fichier exécutable binaire (.exe, .out, etc.) que l'ordinateur peut comprendre directement. Une fois compilé, ce fichier exécutable peut être lancé sans avoir besoin de l'outil de compilation.

Exemples de langages compilés :

- C
- C++
- Go
- Rust

Avantages des langages compilés :

- Les programmes compilés sont généralement plus rapides à l'exécution, car ils sont directement convertis en code machine.
- Le programme n'a plus besoin du compilateur pour être exécuté, ce qui facilite la distribution d'une application (un fichier exécutable est suffisant).

Inconvénients :

- Chaque modification du code nécessite une recompilation.
- Il peut être plus complexe de déboguer le code une fois compilé.

Langages interprétés

Les **langages interprétés** sont exécutés ligne par ligne par un interpréteur, sans passer par une phase de compilation préalable. Lorsque vous lancez un programme écrit dans un langage interprété, l'interpréteur lit chaque ligne du code, le traduit en instructions machine, et l'exécute.

Ouvrir le notebook

Exemples > Recherchez une URL GitHub ou effectuez une recherche par organisation ou par <https://github.com/enicomet/python-lut> ☐ Inclure les dépôts privés

Récents >

Dépot: ☐ Branche:

Google Drive >

GitHub >

Importer >

Python_LUT_INNO3.py

71.17.028 (rapporté)

0 s terminée à 17:55